

FREE HANDS-ON COURSE · EXPLAINED VISUALLY

Docker, from beginner to advanced: every command tested

116 lessons where you run it, break it on purpose, read the real error and fix it.

116

hands-on lessons

25

troubleshooting problems

1,777

code blocks tested in CI

8 + 1

projects + a capstone

THE REAL PROBLEM

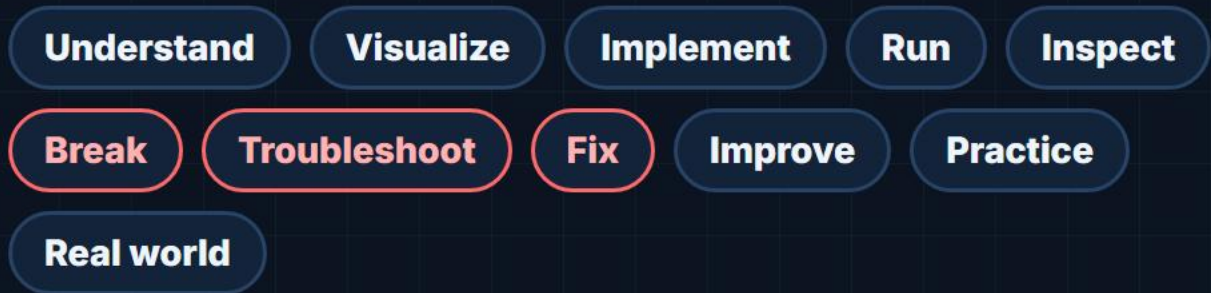
Tutorials end at **docker run.**
Work starts with these:

```
Bind for 0.0.0.0:8080 failed: port is already allocated
report: Exited (137) 2 seconds ago
exit code: 137, OOMKilled: true
touch: cannot touch '/app/cache.tmp': Read-only file system
wget: bad address 'api'
menu-api: Up 4 seconds (unhealthy)
exec: "python": executable file not found in $PATH
```

Real errors from the course's lessons and troubleshooting problems. Each one is reproduced, investigated and fixed.

THE METHOD

Every concept, the same loop



Each lesson has a lab, an intentional failure with its real error output, the investigation, the fix, a challenge with a separate solution, and a recap. Each module ends with an assessment.

CONCEPT · IMAGES ARE LAYERS

Order your Dockerfile for the cache

`COPY . .` ← changes often: last

`RUN npm ci` ← reused while `package*.json` is unchanged

`COPY package*.json ./`

`WORKDIR /app`

`FROM node:24-alpine`

A change invalidates its own layer and every layer above it.
Code last, dependencies first.

The same Go API, measured

single stage



98.0 MB

distroless



3.3 MB

scratch



2.5 MB

Compile in the toolchain image, ship only the binary: no compiler, no shell, no package manager to patch or attack. (Content size, as a pull downloads it.)

Talk by name, publish only the front door

host :8080

-p 8080:8080
only the proxy

proxy → api

http://api:8080
Docker's DNS

api → db

internal network
no way in, no way out

```
$ docker compose exec proxy nslookup db
** server can't find db: NXDOMAIN
# the proxy is not on the database's network: by design
```

Not "should be secure": checked

```
$ docker inspect ... user / read-only / capabilities
proxy: user=nginx read-only=true cap_drop=[ALL] cap_add=[]
frontend: user=nginx read-only=true cap_drop=[ALL] cap_add=[]
api: user=nonroot:nonroot read-only=true cap_drop=[ALL] cap_add=[]
$ docker compose exec proxy sh -c 'echo hacked >
/usr/share/nginx/html/index.html'
sh: can't create /usr/share/nginx/html/index.html: Read-only file
system
$ docker compose exec api sh
... exec: "sh": executable file not found in $PATH
```

From the capstone: non-root users, read-only file systems, no capabilities, no shell in the API, and the database password as a secret file, never an environment variable.

THE CAPSTONE

A production-style stack, end to end

Nginx proxy

frontend

Go API

PostgreSQL

multi-stage

healthchecks

limits

secrets

internal network

volume

log rotation

```
$ docker compose up -d --wait
proxy: healthy
frontend: healthy
api: healthy
db: healthy
$ docker compose stop db
{"status":"database unavailable"}
$ docker compose start db
{"status":"ok","version":"1.0.0"}
```

Then pushed to a registry and deployed by digest; a GitHub Actions pipeline builds, smoke-tests and publishes the images to GHCR.

WHAT IS INSIDE

Everything to learn it, and prove it

19

module assessments

13

final exam tasks, auto-graded

21

videos, full + silent

152

page study guide PDF

Plus a command reference linked to the lessons, 12 diagrams, a glossary and 28 interview questions.

EVERY COMMAND TESTED

The lessons are the test suite

```
←!— test: fail; contains=port is already allocated;  
output →  
```bash  
docker run -d --name web-new -p 8080:80 nginx:1.30-alpine
2>&1
```
```

A runner executes every block as a learner types it, in a sandbox, and writes the real output back into the lesson. GitHub Actions runs all 1,777 blocks on a fresh Docker 29 engine, and the course also runs on Docker Desktop.

FREE AND OPEN

Start with lesson 001



the course



all my projects

Which Docker error cost you the most time?